

Task 4

Software Security - Basic Overflow

Reward: 15 Points

Mode: Project / Group work (max. 3 students)

Due: 29.06.2025 -- 14:15

Information

- **Remote Exercise Framework (REF)**

Before you start the task, you should read, understand, and follow the instructions for the *Remote Exercise Framework* provided in Moodle.

- **Group Work**

This task can be completed in groups. You may work in a group of up to 3 students (i.e., 1, 2, or 3 people), and your group selection is binding for the remainder of the semester. It is sufficient if one person from the group submits the solution (both the code in REF and a submission PDF in Moodle).

- **Deliverables**

All code submissions are made directly in REF; you can find more details in the REF guide on Moodle. All code must be sufficiently and meaningfully commented. Solutions to pen & paper tasks are uploaded as a single PDF file to Moodle.

Important: We reserve the right to check all submissions for plagiarism and take appropriate action.

Task 4: Data-Only Attack

The following C snippet contains a vulnerability in password authentication that allows successful authentication even with an incorrect password.

```
1 int check_authentication(char *password) {
2     int auth_flag = 0;
3     char password_buffer[12];
4     strcpy(password_buffer, password);
5     if(strcmp(password_buffer, "swordfish") == 0)
6         auth_flag = 1;
7     return auth_flag;
8 }
9
10 int main(int argc, char *argv[]) {
11     if(argc < 2) {
12         printf("Usage: %s <password>\n", argv[0]);
13         exit(0);
14     }
15     char *password = argv[1];
16     if (check_authentication(password)) printf("Correct!\n");
17     else printf("You shall not pass!\n");
18 }
```

- a) Fill out the stack frame layout of the `check_authentication` function and describe each memory region (return address, parameters, local variables, base pointer, ...) as well as the size (left). Then fill in the actual memory contents of the stack after an attack (right). This task is supposed to be solved conceptually and not with the concrete memory values obtained during debugging. Any undefined or unknown values may be marked with '??'. Note that the local variable `password_buffer` starts in the bottom left of the address range.
- b) Analyze the program `doa`, which contains an updated, hardened version of the authentication function. What has changed, and how can this change be bypassed? Briefly explain the new check(s) and find a password that is not `swordfish` that results in a success message. The password is read from the file `solution.txt`.

Note: Up to three additional bytes are still working, it breaks beyond these.

Note: The source code is not available for this task and no longer matches the snippet above, as the function has been hardened.

Task name for remote login: doa

	Description of memory area	Size in Bytes
memory addresses 0xFFFFFFFF ↓ 0x00000000	previous memory	-
	free memory	-

Stackframe after attack			
previous memory			
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
\x	\x	\x	\x
free memory			

0x00000000

memory addresses

0x00000003

→

Task 5: Stack-based Buffer Overflow

The programs `basic_overflow` and `slide_rider` are vulnerable to stack-based buffer overflow attacks. The task is to exploit this and inject the shellcode given in task 5a. Write and submit a *commented* script that prints the exploit to `stdout`, so the output can be used as an argument for the target program. After spawning the shell, you should be able to read the flag in the same directory.

a) Simple Buffer Overflow

First, analyze the `basic_overflow` program. The buffer overflow occurs at line 10, since input length is not checked. Before writing your exploit, determine how many bytes are needed to overwrite the return address and how you will inject shellcode into the process. The program will help you by printing the buffer's address during execution.

```
<SHELLCODE> = "\x31\xc9\xf7\xe1\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\xe\x89\xe3\xb0\x0b\xcd\x80"
```

Note: For this task, there is no task check functionality implemented. Please, however, create a file `solution.sh` that, when run with `/home/user/basic_overflow $(./solution.sh)`, exploits the vulnerability to spawn a shell in which the flag is readable.

Task name for remote login: `basic_overflow`

b) NOP Sliding

As we've seen in previous tasks, memory addresses can change due to factors like environment variables. This can render exploits based on specific addresses useless. A common technique to improve exploit reliability is the so-called NOP-slide — a large sequence of no-operation (NOP) instructions placed before the actual shellcode, serving as a *landing zone* for jumps into the shellcode.

Your task is to inject the shellcode and read the secret flag again — but this time using a NOP-slide so your exploit remains stable even if addresses change slightly. Remember that you can overwrite more than just the current stack frame.

Note: Source code is not available for this task.

Task name for remote login: `slide_rider`

Plagiarism & Use of Generative AI

In this course, we respect other people's work and do not tolerate plagiarism. That means you should always give credit to the original authors if you:

- use someone else's work (whether you copy it directly, almost directly, or rewrite it in your own words),
- reuse parts of someone else's work, such as equations, figures, or tables.

We strongly encourage you to work on this assignment without using generative AI tools (like large language models), so you can build your own understanding of the material. Please, **in all cases**, state clearly how you used such tools:

- If you only used them for small edits (e.g., grammar or wording), add a note like: “This assignment was edited for grammar using [Tool Name].”
- If you did not use any AI tools, add: “This assignment was completed without the use of AI tools.”
- If you use AI for more than small edits, please specify in detail which parts of the solutions are generated by the AI tool and which steps you took to ensure the correctness of the output.
- Work that relies on AI to generate the major part of the submission, e.g., full scripts or key parts of the solution, will not be accepted.

If you use Stack Overflow or similar resources, make sure to clearly cite any code you did not write yourself.